

Class & Object in JAVA - part 2

Pertemuan 3

Dosen Pengampu:

- Zulkifli Arsyad
- Ardhian Ekawijana
- Ghifari Munawar



Last Week

1. Object
2. Class
3. Message
4. Encapsulation



Topics today..

1. Relationship Between Class
2. Static Field & Method
3. Method Parameters
4. Object Construction
5. Packages
6. Jar



1. Relationship Between Class

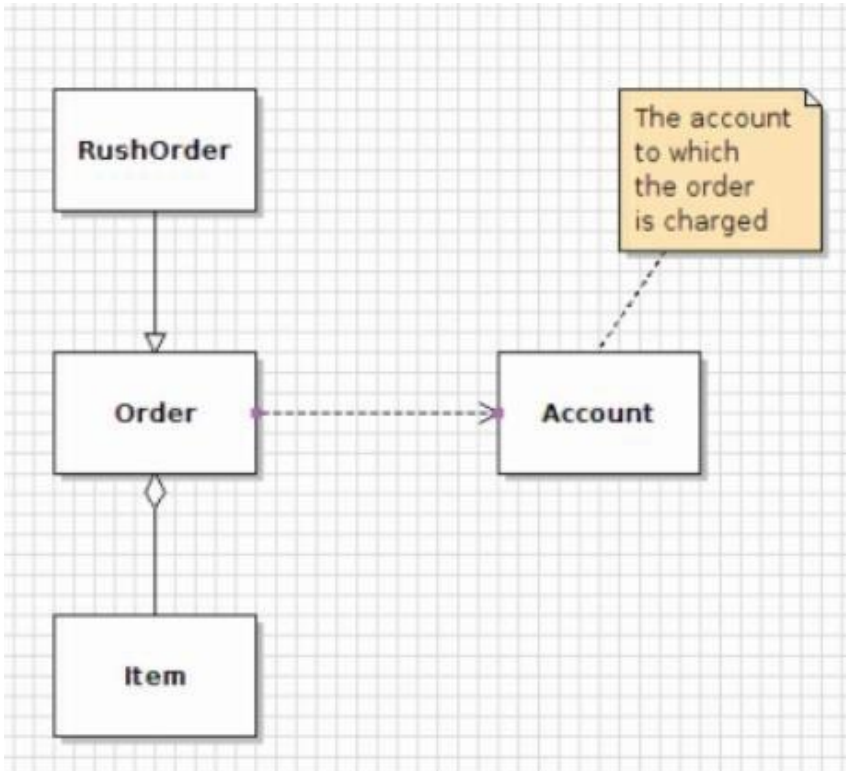


Relationship Between Class

- Dependence (“uses-a”)
- Aggregation (“has-a”)
- Inheritance (“is-a”)



Contoh “Class relationship”



Relationship	UML Connector
Inheritance	
Interface implementation	
<u>Dependency</u>	
Aggregation	
Association	
Directed association	

2. Static Fields & Methods



Static Fields

- Digunakan untuk mendefinisikan field yang bersifat static dan dapat diakses global (diambil dan diset nilainya) oleh semua objek yang merepresentasikan classnya.
- Dapat diakses langsung tanpa membuat object

```
public class Employee {  
    private static String companyName = "ABC Corp";  
  
    public static String getCompanyName() {  
        return companyName;  
    }  
  
    public static void main(String[] args) {  
        // Mengakses static method langsung Lewat class  
        System.out.println(Employee.getCompanyName()); // ABC Corp  
    }  
}
```

Static Fields

3 contoh variable bertipe static

- Konstanta (Constant)

```
public static final double PI = 3.14159;
```

- Counter / Tracker (Menyimpan jumlah atau status global yang berbagi untuk semua objek.)
- `private static int totalEmployees = 0;`
- Shared Configuration / Resource
`private static String databaseUrl = "jdbc:mysql://localhost:3306/mydb";`



Static Constant

- Static constant pada implementasinya lebih sering digunakan dibandingkan dengan static fields.
- Static constant artinya variable bersifat tetap (tidak dapat diubah), namun dapat diakses (diambil nilainya) secara global oleh objek dari kelas tersebut.
- Contoh : dapat dipanggil nilainya melalui **Math.PI**

```
public class Math
{
    ...
    public static final double PI = 3.14159265358979323846;
    ...
}
```



Static Methods

- Static methods merupakan method yang tidak mengoperasikan objek, artinya tidak memiliki implicit parameter.
- Dapat digunakan pada 2 situasi berikut :
 - Ketika method tidak membutuhkan akses ke implicit parameter (field dari suatu objek), tetapi hanya disuplai dari explicit parameter.
 - Ketika method dalam memprosesnya hanya membutuhkan akses ke static fields saja



Contoh static method

```
import java.io.*;
public class Contoh {

    static int num = 100;
    static String str = "Belajar static method";

    // ini static method
    static void iniStatic()
    {
        System.out.println("static number is " + num);
        System.out.println("static string is " + str);
    }

    // ini non-static method
    void nonstatic()
    {
        iniStatic();
    }

    // main method
    public static void main(String args[])
    {
        Contoh obj = new Contoh ();
        // memanggil method non-static harus menggunakan obj
        obj.nonstatic();

        // memanggil method static tidak perlu menggunakan obj
        iniStatic();
    }
}
```



Factory Method

- Method yang digunakan untuk menginstansiasikan objek, namun bukan method constructor.
- Alasan kenapa perlu factory method :
 - Karena method constructor namanya tidak bisa diubah, sehingga agar lebih jelas memberikan nama method sendiri untuk mengkonstruksi objeknya.
 - Ketika anda menggunakan konstruktor, anda tidak dapat bervariasi tipe objek yang dibangun.



Contoh factory method

```
import java.text.*;

public class Contoh {
    // main method

    public static void main(String args[])
    {
        NumberFormat ubahUang = NumberFormat.getCurrencyInstance();
        NumberFormat ubahPersen = NumberFormat.getPercentInstance();

        double x = 0.1;

        System.out.println(ubahUang.format(x));
        System.out.println(ubahPersen.format(x));
    }
}
```



The main method

- Main method merupakan method static yang tidak memiliki implicit parameter. Namun secara umum main method digunakan untuk mengkonstruksi objek.

```
class Employee
{
    public Employee(String n, double s, int year, int month, int day)
    {
        name = n;
        salary = s;
        LocalDate hireDay = LocalDate.now(year, month, day);
    }
    ...
    public static void main(String[] args) // unit test
    {
        Employee e = new Employee("Romeo", 50000, 2003, 3, 31);
        e.raiseSalary(10);
        System.out.println(e.getName() + " " + e.getSalary());
    }
    ...
}
```

3. Method Parameters



Method parameters

- Tipe parameter :
 - By value
 - By reference



4. Object Construction



Object Construction

- Secara umum terdapat 2 cara untuk mengkonstruksi sebuah objek :
 - Dengan memanggil konstruktor dan menyisipkan nilainya melalui parameter
 - Dengan meng-assign value pada saat deklarasi objek
- Namun ada cara yang ketiga, yakni dengan initialization block.



Initialization Block

- Initialization block umumnya hanya men-set nilai awal (default) dari objek saat deklarasi, seperti 0, false atau null.

```
class Employee
{
    private static int nextId;

    private int id;
    private String name;
    private double salary;

    // object initialization block
    {
        id = nextId;
        nextId++;
    }

    public Employee(String n, double s)
    {
        name = n;
        salary = s;
    }

    public Employee()
    {
        name = "";
        salary = 0;
    }
}
```



Constructor

- Constructor adalah method yang secara otomatis dipanggil/dieksekusi saat sebuah class diinstansiasi.
- Nama constructor harus sama dengan nama class-nya.
- Sama halnya dengan method constructor juga dapat memiliki satu atau lebih dari satu parameter.



Contoh implementasi constructor

```
public class Mahasiswa
{
    private String nim, nama;
    public Mahasiswa()
    {
        this.nim = "";
        this.nama = "";
    }
}
```



Contd..

```
public class Mahasiswa
{
    private String nim, nama;
    public Mahasiswa ()
    {
        this.nim="";
        this.nama = "";
    }
    public Mahasiswa(String nim, String nama)
    {
        this.nim = nim;
        this.nama = nama;
    }
}
```

Contoh class yang memiliki lebih dari satu constructor disebut **multiple constructor**



Function Overloading

- Function **Overloading** merupakan suatu kondisi dimana suatu class memiliki fungsi yang sama tetapi deklarasi dan parameternya berbeda.



Contoh implementasi :

```
public class Matematika
{
    ....
    public int Tambah (int a, int b)
    {
        return a + b;
    }
    public int Tambah (double a, double b)
    {
        return a + b;
    }
}
```



5. Package



Package

- Java memungkinkan sekumpulan class dikelompokkan kedalam satu packages.
- Packages dapat meningkatkan konsep modularity, sehingga dapat mengelola kode dengan lebih jelas (tanggungjawabnya).



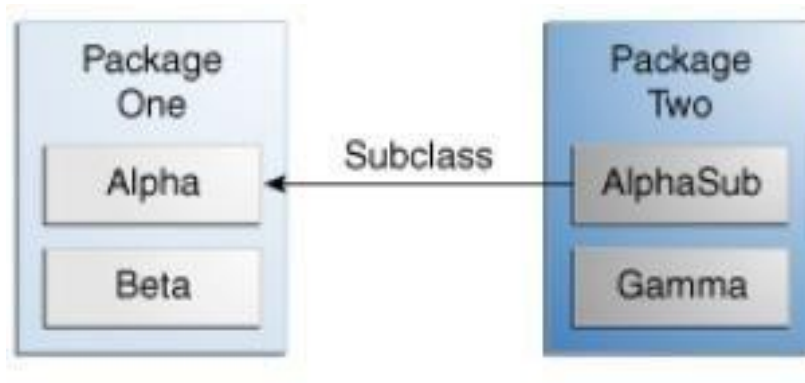
Class Importation

- Sebuah class dapat menggunakan semua kelas pada packagenya dan class public lain dari package yang berbeda.
- Untuk menggunakannya dapat melalui import package
- Kita juga dapat menggunakan notasi * untuk mengambil semua package pada prefix java (contoh : import java.*)

```
import java.util.ArrayList;  
import java.util.Date;
```



Package Scope



Visibility

Modifier	Alpha	Beta	Alphasub	Gamma
<code>public</code>	Y	Y	Y	Y
<code>protected</code>	Y	Y	Y	N
<code>no modifier</code>	Y	Y	N	N
<code>private</code>	Y	N	N	N

Static Imports

- Disamping kelas, kita juga dapat mengimport method / fields yang sifatnya static.

For example, if you add the directive

```
import static java.lang.System.*;
```

to the top of your source file, then you can use the static methods and fields of the `System` class without the class name prefix:

```
out.println("Goodbye, World!"); // i.e., System.out  
exit(0); // i.e., System.exit
```

You can also import a specific method or field:

```
import static java.lang.System.out;
```



Menambahkan kelas pada packages

- Menuliskan package disertai nama pada awal deklarasi kelas

```
package com.horstmann.corejava;  
  
public class Employee  
{  
    . . .  
}
```

Menggunakan Jar File

1. Buka terminal jalankan perintah berikut

```
javac -d bin src/id/ac/polban/employee/model/*.java  
src/id/ac/polban/employee/service/*.java src/App.java
```

2. Buat file manifest.txt

Main-Class: src/App

3. Buat jar

```
jar cfm EmployeeApp.jar src/manifest.txt -C bin .
```

4. Jalankan jar

```
java -jar EmployeeApp.jar
```



Create project untuk membaca jar file

The screenshot shows an IDE with the following components:

- EXPLORER:** Displays the project structure with folders `.vscode`, `bin`, `lib`, and `src`. The `src` folder contains `App.java` and `README.md`.
- Source Editor:** Shows the code for `App.java`. The code imports classes from `id.ac.polban.employee.model` and `id.ac.polban.employee.service`. It defines a `public class App` with a `main` method that initializes `Department` and `EmploymentType` objects, creates `Employee` objects, and calls `service.addEmployee(emp1)`.
- TERMINAL:** Shows the command to run the application: `PS D:\POLBAN\2025\Ganjil 20252026\PBO\Pertemuan 3\employeeJar\employeeJar> & 'D:\APPLICATION\jdk-17.0.10_windows-x64_bin\jdk-17.0.10\bin\java.exe' '@C:\Users\JTK140~1\AppData\Local\Temp\cp_16ynge2pxv58\nkwnnjfuv8k.argfile' 'App'`. The output shows the initial salary of 7000000.0 and the updated salary of 7700000.000000001 after the `kenaikan` (increase) operation.



Kesimpulan :

1. Relationship Between Class
2. Static Field & Method
3. Method Parameters
4. Object Construction
5. Packages



Questions



Latihan

- Jalankan Program
- Build ke dalam Jar file
- Buat project untuk menjalankan jar file
- Buat Diagram kelasnya.



Praktikum

- Lengkapi kasus anda dengan menerapkan static, package
- buat package `id.ac.polban.model` dan `id.ac.polban.service`
- Buat kelas diagram
- Implementasikan relasi antar kelas melalui :
 - Dependency
 - Aggregation
- Jelaskan masing-masing relasi tersebut pada kasus yang dibuat!



Praktikum

- Generate aplikasi anda ke dalam jar file
- buat project baru yang hanya memuat jar file saja.
- Source code berikut penjelasannya dikumpulkan dalam dokumen PDF



References :

1. Core Java Volume 1(10th Edition), Chapter 4 Class & Object

